

Dvarpala VPN Project - Complete Reference Document

Table of Contents

1. [Project Overview](#)
 2. [Architecture Design](#)
 3. [Database Schema](#)
 4. [Authentication Flow](#)
 5. [Project Directory Structure](#)
 6. [Implementation Details](#)
 7. [API Specifications](#)
 8. [Deployment Guide](#)
 9. [Quick Start Commands](#)
 10. [Future Enhancements](#)
-

Project Overview

Project Name: Dvarpala VPN

Etymology: Dvarpala (द्वारपाल) - Sanskrit term meaning "gatekeeper" or "door guardian" from Hindu/Indian mythology. Perfect for a VPN system that controls access to organizational resources.

Core Objectives

Primary Goal: Create a secure, enterprise-grade VPN solution that serves as the first entry point for all organizational dashboards and resources, with granular access control based on user roles and groups.

Key Features:

- OpenVPN-based secure connection
- Multi-provider OAuth authentication (Gmail/Microsoft/GitHub)
- Captive portal authentication flow
- Group-based access control
- Dynamic network route assignment
- Centralized user and resource management
- API for external tool integration
- Audit logging and monitoring

Target Users

- **Developers:** Access to development environments and tools
 - **DevOps:** Infrastructure management and deployment tools
 - **Finance:** Financial dashboards and reporting tools
 - **Management:** Administrative dashboards and analytics
 - **IT Administrators:** System management and user administration
-

Architecture Design

High-Level Architecture



Core Components

1. OpenVPN Server

- **Purpose:** Primary VPN connectivity
- **Configuration:** Two-stage access (captive portal → full access)
- **Authentication:** Custom script integration with Dvarpala backend
- **Network Segmentation:** Dynamic route pushing based on user groups

2. Dvarpala Web Application

- **Framework:** Flask (Python)
- **Database:** PostgreSQL with Redis for session management
- **Authentication:** Multi-provider OAuth (Google, Microsoft, GitHub)
- **Admin Interface:** User, group, and resource management
- **API:** RESTful endpoints for external tool integration

3. Access Control Engine

- **Group-based permissions:** Hierarchical group support
- **Resource management:** Dashboards, VMs, databases, services

- **Dynamic routing:** Network access based on user permissions
- **Audit logging:** Comprehensive access and activity tracking

Security Model

Zero Trust Architecture

- **Principle:** Never trust, always verify
- **Implementation:** Every access request is authenticated and authorized
- **Benefits:** Minimizes attack surface, prevents lateral movement

Multi-Factor Authentication

1. **OpenVPN Certificate:** Device-level authentication
 2. **OAuth Validation:** Identity verification with company email
 3. **Database Authorization:** User must be explicitly added to Dvarpala
 4. **Session Management:** Time-bound access with automatic expiry
-

Database Schema

Core Tables

sql

-- Users table

```
CREATE TABLE users (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    full_name VARCHAR(255),  
    department VARCHAR(100),  
    status ENUM('active', 'inactive', 'suspended') DEFAULT 'active',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    last_login TIMESTAMP,  
    oauth_provider ENUM('gmail', 'microsoft', 'github'),  
    oauth_user_id VARCHAR(255),  
    INDEX idx_email (email),  
    INDEX idx_status (status)  
);
```

-- Groups table with hierarchical support

```
CREATE TABLE groups (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) UNIQUE NOT NULL,  
    description TEXT,  
    parent_group_id BIGINT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (parent_group_id) REFERENCES groups(id),  
    INDEX idx_parent (parent_group_id)  
);
```

-- User-Group mapping

```
CREATE TABLE user_groups (  
    user_id BIGINT,  
    group_id BIGINT,  
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    assigned_by BIGINT,  
    PRIMARY KEY (user_id, group_id),  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
    FOREIGN KEY (group_id) REFERENCES groups(id) ON DELETE CASCADE,  
    FOREIGN KEY (assigned_by) REFERENCES users(id)  
);
```

-- Resources (dashboards, VMs, services)

```
CREATE TABLE resources (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(255) NOT NULL,  
    type ENUM('dashboard', 'vm', 'database', 'service') NOT NULL,  
    url VARCHAR(500),  
    ip_address INET,
```

```
port INT,  
description TEXT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
INDEX idx_type (type),  
INDEX idx_ip (ip_address)  
);
```

-- Group permissions

```
CREATE TABLE group_permissions (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  group_id BIGINT NOT NULL,  
  resource_id BIGINT NOT NULL,  
  permission_type ENUM('read', 'write', 'admin', 'ssh', 'full') NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (group_id) REFERENCES groups(id) ON DELETE CASCADE,  
  FOREIGN KEY (resource_id) REFERENCES resources(id) ON DELETE CASCADE,  
  UNIQUE KEY unique_group_resource (group_id, resource_id)  
);
```

-- VPN Sessions tracking

```
CREATE TABLE vpn_sessions (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  client_ip INET,  
  mac_address VARCHAR(17),  
  connected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  disconnected_at TIMESTAMP NULL,  
  status ENUM('captive', 'authenticated', 'disconnected') DEFAULT 'captive',  
  FOREIGN KEY (user_id) REFERENCES users(id),  
  INDEX idx_user_status (user_id, status),  
  INDEX idx_client_ip (client_ip)  
);
```

-- Audit logs

```
CREATE TABLE audit_logs (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT,  
  action VARCHAR(100) NOT NULL,  
  resource_type VARCHAR(50),  
  resource_id BIGINT,  
  ip_address INET,  
  user_agent TEXT,  
  details JSON,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id),  
  INDEX idx_user_action (user_id, action),
```

```
INDEX idx_created_at (created_at)
);
```

Authentication Flow

Detailed User Experience Flow

Initial Setup (One-time per device)

1. **Admin provides .ovpn file** to user
2. **User installs OpenVPN client** on their device
3. **User imports .ovpn configuration** into OpenVPN client

Daily Connection Flow

Step 1: VPN Connection Attempt

User clicks "Connect" in OpenVPN client

↓

Client sends connection request with static credentials:

- Username: temp_user
- Password: temp_portal_access
- Client certificates (embedded in .ovpn)

Step 2: Temporary Connection Established

OpenVPN Server Response:

↓

Grants limited network access (captive portal mode)

- IP Range: 10.8.0.x (captive portal network)
- Accessible: Only dvarpala.yourcompany.com
- All other traffic: Blocked by firewall rules
- Auto-opens browser: http://dvarpala.yourcompany.com

Step 3: Web Authentication

User sees authentication page with options:

- "Login with Google Workspace"
- "Login with Microsoft 365"
- "Login with GitHub"

↓

User selects their organization's OAuth provider

↓

Redirected to OAuth provider for authentication

↓

After successful OAuth, redirected back to Dvarpala

Step 4: Dual Validation

OAuth Validation:

- ✓ Valid company email domain
- ✓ Active access to email account

↓

Database Validation:

- ✓ User exists in Dvarpala database
- ✓ User status is 'active'
- ✓ User has assigned groups with permissions

Step 5: Full Access Grant

If both validations pass:

↓

Server stores authentication session in Redis

↓

OpenVPN server receives "upgrade access" command

↓

Client automatically reconnects with full network access

- IP Range: 10.8.1.x (full access network)
- Accessible: Resources based on user groups
- Routes: Dynamically pushed based on permissions

Behind-the-Scenes Technical Flow

OpenVPN Authentication Script

python

```
#!/usr/bin/env python3
# /etc/openvpn/dvarpala-auth.py

import os
import json
import redis
import time

def authenticate_user():
    username = os.environ.get('username')
    password = os.environ.get('password')
    client_ip = os.environ.get('untrusted_ip')

    # Always allow initial connection with temp credentials
    if username == "temp_user" and password == "temp_portal_access":
        auth_status = check_user_authorization(client_ip)

        if auth_status == "authorized":
            grant_full_network_access(client_ip)
            return True
        else:
            grant_captive_portal_access(client_ip)
            return True

    return False

def check_user_authorization(client_ip):
    # Check Redis for recent web authentication
    r = redis.Redis(host='localhost', port=6379, db=0)
    session_key = f"auth:{client_ip}"
    session_data = r.get(session_key)

    if not session_data:
        return "pending_oauth"

    session = json.loads(session_data)

    # Check if both OAuth and database validation passed
    if (session.get('oauth_validated') and
        session.get('db_authorized') and
        not session_expired(session['timestamp'])):
        return "authorized"

    return "unauthorized"

def grant_full_network_access(client_ip):
```

```

# Push routes based on user groups
session_data = get_user_session(client_ip)
user_groups = session_data['groups']

# Generate dynamic routes based on group permissions
routes = generate_routes_for_groups(user_groups)
push_routes_to_client(client_ip, routes)

def grant_captive_portal_access(client_ip):
    # Limited access - only authentication portal
    config = """
    ifconfig-push 10.8.0.100 10.8.0.1
    push "route 10.8.1.10 255.255.255.255"
    push "block-outside-dns"
    """
    apply_client_config(client_ip, config)

if __name__ == "__main__":
    exit(0 if authenticate_user() else 1)

```

Web Authentication Handler

python

```
# src/web/auth/oauth_callback.py
```

```
@app.route('/auth/oauth/callback')
```

```
def oauth_callback():
```

```
    # Step 1: Validate OAuth token
```

```
    email = validate_oauth_token(request.args.get('code'))
```

```
    if not email:
```

```
        return render_template('error.html',  
                                message='OAuth validation failed')
```

```
    # Step 2: Check email domain whitelist
```

```
    domain = email.split('@')[1]
```

```
    if domain not in current_app.config['ALLOWED_DOMAINS']:
```

```
        return render_template('error.html',  
                                message='Domain not authorized')
```

```
    # Step 3: Database authorization check
```

```
    user = User.query.filter_by(email=email, status='active').first()
```

```
    if not user:
```

```
        audit_log(email, 'unauthorized_vpn_attempt', request.remote_addr)
```

```
        return render_template('error.html',  
                                message='User not authorized for VPN access')
```

```
    # Step 4: Store complete authorization
```

```
    client_ip = get_client_ip(request)
```

```
    user_groups = get_user_groups(user.id)
```

```
    auth_data = {
```

```
        'oauth_validated': True,
```

```
        'db_authorized': True,
```

```
        'user_id': user.id,
```

```
        'email': email,
```

```
        'groups': [{ 'id': g.id, 'name': g.name } for g in user_groups],
```

```
        'timestamp': time.time()
```

```
    }
```

```
    # Store in Redis with configurable TTL
```

```
    redis_client.setex(f"auth:{client_ip}",  
                       current_app.config['SESSION_DURATION'],  
                       json.dumps(auth_data))
```

```
    # Update user's last login
```

```
    user.last_login = datetime.utcnow()
```

```
    db.session.commit()
```

```
    # Trigger VPN access upgrade
```

```
trigger_vpn_reconnect(client_ip)

return render_template('success.html',
                       message='VPN access granted. Reconnecting...')
```

Security Features

Session Management

- **Session Duration:** Configurable (5 minutes to 8 hours)
- **Session Storage:** Redis with automatic expiry
- **Session Validation:** Every VPN connection checks session validity
- **Forced Reconnection:** Sessions can be revoked in real-time

Failed Authentication Handling

python

```
def handle_failed_authentication(client_ip, email=None):
    # Log the attempt
    audit_log(email or 'unknown', 'failed_vpn_auth', client_ip)

    # Implement progressive blocking
    failed_attempts = increment_failed_attempts(client_ip)

    if failed_attempts >= 3:
        # Block IP for 20 minutes
        block_ip_temporarily(client_ip, duration_minutes=20)

        # Notify administrators
        send_security_alert(f"Multiple failed VPN attempts from {client_ip}")

    # Disconnect current VPN session
    disconnect_vpn_client(client_ip)

def block_ip_temporarily(ip_address, duration_minutes=20):
    # Add iptables rule
    subprocess.run([
        'iptables', '-I', 'INPUT',
        '-s', ip_address,
        '-j', 'DROP'
    ])

    # Schedule automatic removal
    schedule_ip_unblock(ip_address, duration_minutes)
```

Project Directory Structure

Complete Directory Layout

```

dvarpala/
├── README.md                # Project documentation
├── docker-compose.yml       # Local development environment
├── Makefile                 # Build and deployment tasks
├── .env.example             # Environment variables template
├── .gitignore               # Git ignore patterns
├── requirements.txt          # Python dependencies
├── setup.py                 # Package installation script
├──
├── docs/                   # Documentation
│   ├── api/                # API documentation
│   │   ├── authentication.md
│   │   ├── user-management.md
│   │   └── resource-access.md
│   ├── deployment/         # Deployment guides
│   │   ├── docker.md
│   │   ├── kubernetes.md
│   │   └── openvpn-setup.md
│   ├── architecture/       # Architecture diagrams
│   │   ├── system-overview.md
│   │   └── database-schema.md
│   └── user-guides/        # User documentation
│       ├── admin-guide.md
│       └── end-user-guide.md
├──
├── scripts/                # Utility and deployment scripts
│   ├── setup/              # Initial setup scripts
│   │   ├── install-dependencies.sh
│   │   ├── setup-database.sh
│   │   └── configure-openvpn.sh
│   ├── migration/          # Database migration scripts
│   │   ├── 001_initial_schema.sql
│   │   ├── 002_add_groups.sql
│   │   └── migrate.py
│   ├── deployment/         # Deployment scripts
│   │   ├── deploy.sh
│   │   ├── backup.sh
│   │   └── rollback.sh
│   └── monitoring/         # Monitoring scripts
│       ├── health-check.py
│       └── vpn-status.py
├──
├── config/                 # Configuration files
│   ├── environments/       # Environment-specific configs
│   │   ├── development.py
│   │   └── staging.py

```



```

├── production.py
├── openvpn/                                # OpenVPN configuration templates
│   ├── server.conf.template
│   ├── client.ovpn.template
│   └── auth-scripts/
│       ├── dvarpala-auth.py
│       ├── client-connect.sh
│       └── client-disconnect.sh
├── nginx/                                  # Nginx configuration
│   ├── nginx.conf
│   └── sites-available/
│       └── dvarpala.conf
├── logging.conf                            # Logging configuration
├── src/                                    # Main application source code
│   ├── __init__.py
│   ├── app.py                             # Flask application factory
│   ├── wsgi.py                             # WSGI entry point
│   └── core/                               # Core application logic
│       ├── __init__.py
│       ├── config.py                       # Configuration management
│       ├── database.py                     # Database connection and setup
│       ├── redis_client.py                 # Redis connection
│       ├── exceptions.py                   # Custom exceptions
│       └── constants.py                    # Application constants
├── models/                                # Database models (SQLAlchemy)
│   ├── __init__.py
│   ├── base.py                             # Base model class
│   ├── user.py                             # User model
│   ├── group.py                           # Group model
│   ├── resource.py                         # Resource model
│   ├── permission.py                       # Permission model
│   ├── session.py                         # VPN session model
│   └── audit_log.py                        # Audit logging model
├── auth/                                   # Authentication module
│   ├── __init__.py
│   ├── oauth/                              # OAuth providers
│   │   ├── __init__.py
│   │   ├── base.py                         # Base OAuth provider
│   │   ├── google.py                       # Google OAuth implementation
│   │   ├── microsoft.py                    # Microsoft OAuth implementation
│   │   └── github.py                       # GitHub OAuth implementation
│   ├── decorators.py                       # Authentication decorators
│   └── middleware.py                       # Authentication middleware

```

```

└─ utils.py                # Authentication utilities

└─ vpn/                    # VPN management module
    └─ __init__.py
    └─ openvpn_manager.py  # OpenVPN server management
    └─ client_manager.py   # VPN client session management
    └─ network_manager.py  # Network access control
    └─ certificate_manager.py # SSL certificate management
    └─ session_manager.py  # VPN session lifecycle

└─ access_control/        # Access control module
    └─ __init__.py
    └─ permissions.py     # Permission checking logic
    └─ groups.py          # Group management logic
    └─ resources.py       # Resource management logic
    └─ policy_engine.py   # Access policy evaluation

└─ api/                   # API endpoints
    └─ __init__.py
    └─ v1/                # API version 1
        └─ __init__.py
        └─ auth.py        # Authentication endpoints
        └─ users.py       # User management endpoints
        └─ groups.py      # Group management endpoints
        └─ resources.py   # Resource management endpoints
        └─ permissions.py # Permission management endpoints
        └─ vpn.py         # VPN management endpoints
        └─ external.py    # External tool integration
    └─ middleware.py      # API middleware
    └─ validators.py      # Request validators

└─ web/                   # Web interface
    └─ __init__.py
    └─ auth/              # Authentication web routes
        └─ __init__.py
        └─ login.py
        └─ oauth_callback.py
        └─ logout.py
    └─ admin/             # Admin dashboard routes
        └─ __init__.py
        └─ dashboard.py
        └─ users.py
        └─ groups.py
        └─ resources.py
        └─ permissions.py
    └─ user/              # User dashboard routes
        └─ __init__.py

```

```

| | | | └─ profile.py
| | | | └─ vpn_status.py
| | | └─ forms/ # WTFORMs for web interface
| | | | └─ __init__.py
| | | | └─ user_forms.py
| | | | └─ group_forms.py
| | | | └─ resource_forms.py
| | └─ services/ # Business logic services
| | | └─ __init__.py
| | | └─ user_service.py # User management business logic
| | | └─ group_service.py # Group management business logic
| | | └─ resource_service.py # Resource management business logic
| | | └─ permission_service.py # Permission management business logic
| | | └─ vpn_service.py # VPN management business logic
| | | └─ audit_service.py # Audit logging business logic
| | | └─ notification_service.py # Notification business logic
| | └─ utils/ # Utility functions
| | | └─ __init__.py
| | | └─ encryption.py # Encryption and security utilities
| | | └─ validators.py # Data validation utilities
| | | └─ formatters.py # Data formatting utilities
| | | └─ email.py # Email utilities
| | | └─ csv_parser.py # CSV parsing utilities
| | └─ cli/ # Command line interface
| | | └─ __init__.py
| | | └─ commands/ # CLI command implementations
| | | | └─ __init__.py
| | | | └─ user.py # User management commands
| | | | └─ group.py # Group management commands
| | | | └─ vpn.py # VPN management commands
| | | | └─ admin.py # Administrative commands
| | | └─ cli.py # CLI entry point
| └─ templates/ # Jinja2 HTML templates
| | └─ base.html # Base template with common layout
| | └─ auth/ # Authentication templates
| | | └─ login.html
| | | └─ oauth_login.html
| | | └─ success.html
| | | └─ error.html
| | └─ admin/ # Admin dashboard templates
| | | └─ base.html
| | | └─ dashboard.html
| | | └─ users/

```

[illegible]

- | |─ conftest.py # Pytest configuration and fixtures
- | |─ unit/ # Unit tests
 - | | |─ __init__.py
 - | | |─ test_models.py
 - | | |─ test_auth.py
 - | | |─ test_services.py
 - | | |─ test_utils.py
- | |─ integration/ # Integration tests
 - | | |─ __init__.py
 - | | |─ test_api.py
 - | | |─ test_vpn.py
 - | | |─ test_oauth.py
- | |─ e2e/ # End-to-end tests
 - | | |─ __init__.py
 - | | |─ test_user_flow.py
 - | | |─ test_admin_flow.py
- | |─ fixtures/ # Test data fixtures
 - | | |─ users.json
 - | | |─ groups.json
 - | | |─ test_data.sql
- |─ migrations/ # Database migrations (Alembic)
 - | |─ versions/ # Migration version files
 - | |─ alembic.ini # Alembic configuration
 - | |─ env.py # Migration environment setup
 - | |─ script.py.mako # Migration script template
- |─ monitoring/ # Monitoring and observability
 - | |─ prometheus/ # Prometheus monitoring
 - | | |─ prometheus.yml
 - | | |─ rules/
 - | | | |─ dvarpala.yml
 - | |─ grafana/ # Grafana dashboards
 - | | |─ dashboards/
 - | | | |─ vpn-overview.json
 - | | | |─ user-activity.json
 - | | | |─ system-health.json
 - | | |─ provisioning/
 - | |─ logs/ # Log aggregation configuration
 - | | |─ logstash.conf
 - | | |─ filebeat.yml
- |─ deployment/ # Deployment configurations
 - | |─ docker/ # Docker configurations
 - | | |─ Dockerfile.web
 - | | |─ Dockerfile.worker
 - | | |─ docker-compose.prod.yml

```
| | └─ kubernetes/                # Kubernetes manifests
| |   | └─ namespace.yaml
| |   | └─ configmap.yaml
| |   | └─ secret.yaml
| |   | └─ deployment.yaml
| |   | └─ service.yaml
| |   | └─ ingress.yaml
| | └─ terraform/                # Infrastructure as Code
| |   | └─ main.tf
| |   | └─ variables.tf
| |   | └─ outputs.tf
| |   | └─ modules/
| |   |   | └─ vpc/
| |   |   | └─ rds/
| |   |   | └─ eks/
| | └─ ansible/                  # Configuration management
| |   | └─ playbooks/
| |   | └─ roles/
| |   | └─ inventory/
| └─ tools/                      # Development tools
|   | └─ linting/                # Code quality tools
|   |   | └─ .flake8
|   |   | └─ .pylintrc
|   |   | └─ pyproject.toml
|   | └─ security/              # Security scanning tools
|   |   | └─ bandit.yaml
|   |   | └─ safety.txt
|   | └─ development/           # Development utilities
|   |   | └─ seed_data.py
|   |   | └─ create_test_users.py
|   |   | └─ generate_certs.py
```

Implementation Details

Core Configuration Files

Flask Application Factory

python

```
# src/app.py
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
from flask_login import LoginManager
from src.core.config import get_config
from src.core.database import init_db
from src.core.redis_client import init_redis

def create_app(config_name='development'):
    app = Flask(__name__)

    # Load configuration
    config = get_config(config_name)
    app.config.from_object(config)

    # Initialize extensions
    init_db(app)
    init_redis(app)

    # Register blueprints
    from src.api.v1 import api_v1_bp
    from src.web.auth import auth_bp
    from src.web.admin import admin_bp
    from src.web.user import user_bp

    app.register_blueprint(api_v1_bp, url_prefix='/api/v1')
    app.register_blueprint(auth_bp, url_prefix='/auth')
    app.register_blueprint(admin_bp, url_prefix='/admin')
    app.register_blueprint(user_bp, url_prefix='/user')

    # Error handlers
    @app.errorhandler(404)
    def not_found(error):
        return render_template('errors/404.html'), 404

    @app.errorhandler(500)
    def internal_error(error):
        return render_template('errors/500.html'), 500

    return app
```

Environment Configuration

python

```
# config/environments/production.py
```

```
import os
```

```
class ProductionConfig:
```

```
    SECRET_KEY = os.environ.get('SECRET_KEY')
```

```
    # Database
```

```
    SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL')
```

```
    SQLALCHEMY_TRACK_MODIFICATIONS = False
```

```
    # Redis
```

```
    REDIS_URL = os.environ.get('REDIS_URL', 'redis://localhost:6379/0')
```

```
    # OAuth Configuration
```

```
    GOOGLE_CLIENT_ID = os.environ.get('GOOGLE_CLIENT_ID')
```

```
    GOOGLE_CLIENT_SECRET = os.environ.get('GOOGLE_CLIENT_SECRET')
```

```
    MICROSOFT_CLIENT_ID = os.environ.get('MICROSOFT_CLIENT_ID')
```

```
    MICROSOFT_CLIENT_SECRET = os.environ.get('MICROSOFT_CLIENT_SECRET')
```

```
    GITHUB_CLIENT_ID = os.environ.get('GITHUB_CLIENT_ID')
```

```
    GITHUB_CLIENT_SECRET = os.environ.get('GITHUB_CLIENT_SECRET')
```

```
    # Allowed email domains
```

```
    ALLOWED_DOMAINS = ['yourcompany.com', 'subsidiary.com']
```

```
    # VPN Configuration
```

```
    OPENVPN_MANAGEMENT_HOST = os.environ.get('OPENVPN_MGMT_HOST', 'localhost')
```

```
    OPENVPN_MANAGEMENT_PORT = int(os.environ.get('OPENVPN_MGMT_PORT', '7505'))
```

```
    # Session Configuration
```

```
    SESSION_DURATION = int(os.environ.get('SESSION_DURATION', '28800')) # 8 hours
```

```
    CAPTIVE_PORTAL_TIMEOUT = int(os.environ.get('CAPTIVE_TIMEOUT', '300')) # 5 minutes
```

```
    # Security
```

```
    FAILED_LOGIN_THRESHOLD = int(os.environ.get('FAILED_LOGIN_THRESHOLD', '3'))
```

```
    IP_BLOCK_DURATION = int(os.environ.get('IP_BLOCK_DURATION', '1200')) # 20 minutes
```

```
    # Logging
```

```
    LOG_LEVEL = os.environ.get('LOG_LEVEL', 'INFO')
```

```
    LOG_FILE = os.environ.get('LOG_FILE', '/var/log/dvarpala/app.log')
```

OpenVPN Server Configuration Template

bash

```
# config/openvpn/server.conf.template
# OpenVPN Server Configuration for Dvarpala

# Basic settings
port 1194
proto udp
dev tun
ca /etc/openvpn/ssl/ca.crt
cert /etc/openvpn/ssl/server.crt
key /etc/openvpn/ssl/server.key
dh /etc/openvpn/ssl/dh2048.pem

# Network configuration
# Captive portal network
server 10.8.0.0 255.255.255.0
# Full access network
topology subnet

# Client configuration
client-config-dir /etc/openvpn/ccd
client-to-client

# Authentication
auth-user-pass-verify /etc/openvpn/auth-scripts/dvarpala-auth.py via-env
script-security 2
username-as-common-name

# Client management
client-connect /etc/openvpn/auth-scripts/client-connect.sh
client-disconnect /etc/openvpn/auth-scripts/client-disconnect.sh

# Management interface
management localhost 7505

# Security
cipher AES-256-CBC
auth SHA256
tls-auth /etc/openvpn/ssl/ta.key 0

# Logging
log /var/log/openvpn/server.log
verb 3
mute 20

# Performance
keepalive 10 120
```

```
comp-lzo  
persist-key  
persist-tun
```

```
# User privilege downgrade  
user nobody  
group nogroup
```

Service Layer Implementation

User Service

python

```

# src/services/user_service.py
from typing import List, Optional, Dict
from src.models.user import User
from src.models.group import Group
from src.models.user_groups import UserGroup
from src.core.database import db
from src.utils.validators import validate_email
from src.services.audit_service import AuditService
import pandas as pd

class UserService:

    @staticmethod
    def create_user(email: str, full_name: str, department: str,
                   oauth_provider: str = None) -> User:
        """Create a new user"""
        if not validate_email(email):
            raise ValueError("Invalid email format")

        # Check if user already exists
        existing_user = User.query.filter_by(email=email).first()
        if existing_user:
            raise ValueError("User already exists")

        user = User(
            email=email,
            full_name=full_name,
            department=department,
            oauth_provider=oauth_provider,
            status='active'
        )

        db.session.add(user)
        db.session.commit()

        # Log the creation
        AuditService.log_action(
            user_id=user.id,
            action='user_created',
            details={'email': email, 'department': department}
        )

        return user

    @staticmethod
    def bulk_create_users(csv_data: str, created_by: int) -> Dict:

```

```

"""Bulk create users from CSV data"""
try:
    df = pd.read_csv(csv_data)
    required_columns = ['email', 'full_name', 'department']

    if not all(col in df.columns for col in required_columns):
        raise ValueError("Missing required columns: email, full_name, dep:

created_users = []
errors = []

for index, row in df.iterrows():
    try:
        # Validate email
        if not validate_email(row['email']):
            errors.append(f"Row {index + 1}: Invalid email format")
            continue

        # Check if user exists
        if User.query.filter_by(email=row['email']).first():
            errors.append(f"Row {index + 1}: User {row['email']} already exists")
            continue

        user = User(
            email=row['email'].strip().lower(),
            full_name=row['full_name'].strip(),
            department=row['department'].strip(),
            status='active'
        )

        created_users.append(user)

    except Exception as e:
        errors.append(f"Row {index + 1}: {str(e)}")

# Bulk insert successful users
if created_users:
    db.session.bulk_save_objects(created_users)
    db.session.commit()

# Log bulk creation
AuditService.log_action(
    user_id=created_by,
    action='bulk_user_creation',
    details={'count': len(created_users), 'errors_count': len(errors)}
)

```

```

        return {
            'created_count': len(created_users),
            'error_count': len(errors),
            'errors': errors
        }

    except Exception as e:
        db.session.rollback()
        raise ValueError(f"Failed to process CSV: {str(e)}")

@staticmethod
def get_user_permissions(user_id: int) -> List[Dict]:
    """Get all permissions for a user through their groups"""
    user = User.query.get(user_id)
    if not user:
        raise ValueError("User not found")

    # Get user's groups (including parent groups)
    user_groups = UserService.get_user_groups_with_inheritance(user_id)

    # Get all permissions for these groups
    from src.models.group_permission import GroupPermission
    permissions = db.session.query(GroupPermission).filter(
        GroupPermission.group_id.in_([g.id for g in user_groups])
    ).all()

    return [
        {
            'resource_id': p.resource_id,
            'resource_name': p.resource.name,
            'resource_type': p.resource.type,
            'permission_type': p.permission_type,
            'group_name': p.group.name
        }
        for p in permissions
    ]

@staticmethod
def get_user_groups_with_inheritance(user_id: int) -> List[Group]:
    """Get user's groups including inherited parent groups"""
    # Direct groups
    direct_groups = db.session.query(Group).join(UserGroup).filter(
        UserGroup.user_id == user_id
    ).all()

    all_groups = set(direct_groups)

```

```

# Add parent groups (inheritance)
for group in direct_groups:
    parent_groups = UserService._get_parent_groups(group)
    all_groups.update(parent_groups)

return list(all_groups)

@staticmethod
def _get_parent_groups(group: Group) -> List[Group]:
    """Recursively get all parent groups"""
    parent_groups = []
    current_group = group

    while current_group.parent_group_id:
        parent_group = Group.query.get(current_group.parent_group_id)
        if parent_group and parent_group not in parent_groups:
            parent_groups.append(parent_group)
            current_group = parent_group
        else:
            break

    return parent_groups

@staticmethod
def assign_user_to_group(user_id: int, group_id: int, assigned_by: int):
    """Assign a user to a group"""
    # Check if assignment already exists
    existing = UserGroup.query.filter_by(
        user_id=user_id,
        group_id=group_id
    ).first()

    if existing:
        raise ValueError("User is already assigned to this group")

    assignment = UserGroup(
        user_id=user_id,
        group_id=group_id,
        assigned_by=assigned_by
    )

    db.session.add(assignment)
    db.session.commit()

    # Log the assignment
    user = User.query.get(user_id)
    group = Group.query.get(group_id)

```



```

        AuditService.log_action(
            user_id=assigned_by,
            action='user_group_assigned',
            details={
                'target_user': user.email,
                'group': group.name,
                'user_id': user_id,
                'group_id': group_id
            }
        )

    @staticmethod
    def deactivate_user(user_id: int, deactivated_by: int):
        """Deactivate a user"""
        user = User.query.get(user_id)
        if not user:
            raise ValueError("User not found")

        user.status = 'inactive'
        db.session.commit()

        # Revoke any active VPN sessions
        from src.services.vpn_service import VPNService
        VPNService.disconnect_user_sessions(user.email)

        # Log deactivation
        AuditService.log_action(
            user_id=deactivated_by,
            action='user_deactivated',
            details={'target_user': user.email, 'user_id': user_id}
        )

```

VPN Service

python

```

# src/services/vpn_service.py
from typing import List, Dict, Optional
import socket
import json
import time
from src.models.vpn_session import VPNSession
from src.models.user import User
from src.core.database import db
from src.core.redis_client import get_redis_client
from src.services.user_service import UserService
from src.services.audit_service import AuditService

class VPNService:

    def __init__(self):
        self.redis_client = get_redis_client()
        self.openvpn_mgmt_host = current_app.config['OPENVPN_MANAGEMENT_HOST']
        self.openvpn_mgmt_port = current_app.config['OPENVPN_MANAGEMENT_PORT']

    def authenticate_vpn_user(self, client_ip: str, username: str, password: str)
        """Authenticate VPN user – called by OpenVPN auth script"""
        try:
            # Always allow temp credentials for captive portal
            if username == "temp_user" and password == "temp_portal_access":
                auth_status = self._check_user_authorization(client_ip)

                if auth_status == "authorized":
                    self._grant_full_network_access(client_ip)
                    return True
                else:
                    self._grant_captive_portal_access(client_ip)
                    return True

            return False

        except Exception as e:
            AuditService.log_action(
                action='vpn_auth_error',
                details={'client_ip': client_ip, 'error': str(e)}
            )
            return False

    def _check_user_authorization(self, client_ip: str) -> str:
        """Check if user has completed web authentication"""
        session_key = f"auth:{client_ip}"
        session_data = self.redis_client.get(session_key)

```

```

if not session_data:
    return "pending_oauth"

try:
    session = json.loads(session_data)

    # Check if both OAuth and database validation passed
    if (session.get('oauth_validated') and
        session.get('db_authorized') and
        not self._session_expired(session.get('timestamp', 0))):
        return "authorized"

    return "unauthorized"

except (json.JSONDecodeError, KeyError):
    return "invalid_session"

def _session_expired(self, timestamp: float) -> bool:
    """Check if authentication session has expired"""
    session_duration = current_app.config['SESSION_DURATION']
    return (time.time() - timestamp) > session_duration

def _grant_full_network_access(self, client_ip: str):
    """Grant full network access based on user groups"""
    session_data = self._get_user_session(client_ip)
    if not session_data:
        return

    user_groups = session_data.get('groups', [])

    # Generate routes based on group permissions
    routes = self._generate_routes_for_groups(user_groups)

    # Apply routes to client
    self._push_routes_to_client(client_ip, routes)

    # Log successful authentication
    AuditService.log_action(
        user_id=session_data.get('user_id'),
        action='vpn_full_access_granted',
        details={'client_ip': client_ip, 'groups': [g['name'] for g in user_g
    )

def _grant_captive_portal_access(self, client_ip: str):
    """Grant limited captive portal access"""
    # Apply captive portal network configuration

```

```

config = {
    'network': '10.8.0.0/24',
    'routes': ['10.8.1.10/32'], # Only auth server
    'dns_block': True
}

self._apply_client_config(client_ip, config)

AuditService.log_action(
    action='vpn_captive_portal_granted',
    details={'client_ip': client_ip}
)

def _generate_routes_for_groups(self, user_groups: List[Dict]) -> List[str]:
    """Generate network routes based on user's group permissions"""
    from src.models.group_permission import GroupPermission
    from src.models.resource import Resource

    routes = []
    group_ids = [g['id'] for g in user_groups]

    # Get all resources user can access
    permissions = db.session.query(GroupPermission).filter(
        GroupPermission.group_id.in_(group_ids)
    ).all()

    for permission in permissions:
        resource = permission.resource

        if resource.ip_address:
            # Add route for specific resource
            routes.append(f"route {resource.ip_address} 255.255.255.255")

        # Add network ranges based on resource type
        if resource.type == 'dashboard':
            # Allow web traffic to dashboard servers
            routes.append(f"route {resource.ip_address} 255.255.255.255")
        elif resource.type == 'vm' and permission.permission_type in ['ssh',
            # Allow SSH access to VMs
            routes.append(f"route {resource.ip_address} 255.255.255.255")

    # Remove duplicates
    return list(set(routes))

def _push_routes_to_client(self, client_ip: str, routes: List[str]):
    """Push network routes to OpenVPN client"""
    try:

```

```

mgmt_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mgmt_socket.connect((self.openvpn_mgmt_host, self.openvpn_mgmt_port))

# Find client session
mgmt_socket.send(b"status\n")
response = mgmt_socket.recv(4096).decode('utf-8')

# Parse client list and find our client
client_id = self._find_client_id_by_ip(response, client_ip)

if client_id:
    # Push routes to specific client
    for route in routes:
        command = f"client-kill {client_id}\n"
        mgmt_socket.send(command.encode('utf-8'))

mgmt_socket.close()

except Exception as e:
    print(f"Failed to push routes: {str(e)}")

def _find_client_id_by_ip(self, status_response: str, target_ip: str) -> Optional[str]:
    """Find OpenVPN client ID by IP address"""
    lines = status_response.split('\n')
    for line in lines:
        if target_ip in line and 'CLIENT_LIST' in line:
            parts = line.split(',')
            if len(parts) > 1:
                return parts[1] # Client ID
    return None

def store_web_authentication(self, email: str, client_ip: str,
                             oauth_provider: str) -> bool:
    """Store successful web authentication"""
    try:
        # Get user from database
        user = User.query.filter_by(email=email, status='active').first()
        if not user:
            return False

        # Get user groups
        user_groups = UserService.get_user_groups_with_inheritance(user.id)

        # Store authentication session
        auth_data = {
            'oauth_validated': True,
            'db_authorized': True,

```

```

        'user_id': user.id,
        'email': email,
        'oauth_provider': oauth_provider,
        'groups': [{'id': g.id, 'name': g.name} for g in user_groups],
        'timestamp': time.time()
    }

    session_key = f"auth:{client_ip}"
    session_duration = current_app.config['SESSION_DURATION']

    self.redis_client.setex(
        session_key,
        session_duration,
        json.dumps(auth_data)
    )

    # Update user's last login
    user.last_login = datetime.utcnow()
    db.session.commit()

    # Create VPN session record
    vpn_session = VPNSession(
        user_id=user.id,
        client_ip=client_ip,
        status='authenticated'
    )
    db.session.add(vpn_session)
    db.session.commit()

    # Log successful authentication
    AuditService.log_action(
        user_id=user.id,
        action='web_authentication_success',
        details={'client_ip': client_ip, 'oauth_provider': oauth_provider}
    )

    return True

except Exception as e:
    AuditService.log_action(
        action='web_authentication_error',
        details={'email': email, 'client_ip': client_ip, 'error': str(e)}
    )
    return False

def disconnect_user_sessions(self, email: str):
    """Disconnect all VPN sessions for a user"""

```

```

user = User.query.filter_by(email=email).first()
if not user:
    return

# Get active sessions
active_sessions = VPNSession.query.filter_by(
    user_id=user.id,
    status='authenticated'
).all()

for session in active_sessions:
    try:
        # Revoke Redis session
        session_key = f"auth:{session.client_ip}"
        self.redis_client.delete(session_key)

        # Disconnect from OpenVPN
        self._disconnect_client_by_ip(session.client_ip)

        # Update session status
        session.status = 'disconnected'
        session.disconnected_at = datetime.utcnow()

    except Exception as e:
        print(f"Failed to disconnect session {session.id}: {str(e)}")

db.session.commit()

AuditService.log_action(
    user_id=user.id,
    action='user_sessions_disconnected',
    details={'session_count': len(active_sessions)}
)

def _disconnect_client_by_ip(self, client_ip: str):
    """Disconnect specific client from OpenVPN"""
    try:
        mgmt_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        mgmt_socket.connect((self.openvpn_mgmt_host, self.openvpn_mgmt_port))

        # Get client list
        mgmt_socket.send(b"status\n")
        response = mgmt_socket.recv(4096).decode('utf-8')

        # Find and disconnect client
        client_id = self._find_client_id_by_ip(response, client_ip)
        if client_id:

```



```

        command = f"client-kill {client_id}\n"
        mgmt_socket.send(command.encode('utf-8'))

    mgmt_socket.close()

except Exception as e:
    print(f"Failed to disconnect client {client_ip}: {str(e)}")

def get_active_sessions(self) -> List[Dict]:
    """Get all active VPN sessions"""
    sessions = db.session.query(VPNSession).join(User).filter(
        VPNSession.status == 'authenticated'
    ).all()

    return [
        {
            'id': s.id,
            'user_email': s.user.email,
            'user_name': s.user.full_name,
            'client_ip': str(s.client_ip),
            'connected_at': s.connected_at.isoformat(),
            'duration': (datetime.utcnow() - s.connected_at).total_seconds()
        }
        for s in sessions
    ]

```

API Specifications

Authentication Endpoints

POST /api/v1/auth/validate

Purpose: Validate user access for external tools

Request:

```

json
{
  "email": "user@yourcompany.com",
  "resource": "developer-dashboard",
  "access_type": "read"
}

```

Response:

```
json
{
  "authorized": true,
  "user_id": 123,
  "groups": ["developers", "backend-team"],
  "permissions": ["read", "write"],
  "expires_at": "2025-06-16T18:30:00Z"
}
```

POST /api/v1/auth/ssh/authorize

Purpose: Authorize SSH access to VMs

Request:

```
json
{
  "email": "user@yourcompany.com",
  "server_ip": "192.168.1.10",
  "ssh_key_fingerprint": "SHA256:abc123..."
}
```

Response:

```
json
{
  "authorized": true,
  "allowed_commands": ["sudo systemctl status", "docker ps"],
  "session_timeout": 3600,
  "audit_required": true
}
```

User Management Endpoints

GET /api/v1/users

Purpose: List all users with pagination

Parameters:

- `page`: Page number (default: 1)
- `per_page`: Items per page (default: 20)
- `status`: Filter by status (active/inactive/suspended)

- `department`: Filter by department
- `search`: Search by name or email

Response:

json

```
{
  "users": [
    {
      "id": 123,
      "email": "john@yourcompany.com",
      "full_name": "John Doe",
      "department": "Engineering",
      "status": "active",
      "last_login": "2025-06-16T10:30:00Z",
      "groups": ["developers", "senior-devs"]
    }
  ],
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total": 150,
    "pages": 8
  }
}
```

POST /api/v1/users/bulk

Purpose: Bulk create users from CSV

Request:

json

```
{
  "csv_data": "email,full_name,department\njohn@company.com,John Doe,Engineering",
  "send_notifications": true
}
```

Response:

json

```
{
  "created_count": 45,
  "error_count": 2,
  "errors": [
    "Row 12: Invalid email format",
    "Row 25: User already exists"
  ]
}
```

Group Management Endpoints

POST /api/v1/groups

Purpose: Create a new group

Request:

json

```
{
  "name": "mobile-developers",
  "description": "Mobile app development team",
  "parent_group_id": 5,
  "permissions": [
    {
      "resource_id": 10,
      "permission_type": "full"
    }
  ]
}
```

GET /api/v1/groups/{group_id}/users

Purpose: Get all users in a group

Response:

json

```
{
  "group": {
    "id": 5,
    "name": "developers",
    "description": "Software developers"
  },
  "users": [
    {
      "id": 123,
      "email": "john@company.com",
      "full_name": "John Doe",
      "assigned_at": "2025-01-15T09:00:00Z"
    }
  ],
  "inherited_users": [
    {
      "id": 124,
      "email": "jane@company.com",
      "full_name": "Jane Smith",
      "inherited_from": "senior-developers"
    }
  ]
}
```

VPN Management Endpoints

GET /api/v1/vpn/sessions

Purpose: Get active VPN sessions

Response:

json

```
{
  "sessions": [
    {
      "id": 456,
      "user_email": "john@company.com",
      "client_ip": "10.8.1.50",
      "connected_at": "2025-06-16T09:30:00Z",
      "duration": 7200,
      "status": "authenticated"
    }
  ],
  "total_active": 23
}
```

POST /api/v1/vpn/disconnect

Purpose: Disconnect specific user sessions

Request:

json

```
{
  "email": "user@company.com",
  "reason": "Security policy violation"
}
```

Response:

json

```
{
  "disconnected_sessions": 2,
  "message": "User sessions disconnected successfully"
}
```

Deployment Guide

Docker Development Setup

docker-compose.yml

yaml

version: '3.8'

services:

web:

build:

context: .

dockerfile: deployment/docker/Dockerfile.web

ports:

- "5000:5000"

environment:

- FLASK_ENV=development

- DATABASE_URL=postgresql://dvarpala:password@postgres:5432/dvarpala

- REDIS_URL=redis://redis:6379/0

volumes:

- ./app

- ./logs:/var/log/dvarpala

depends_on:

- postgres

- redis

postgres:

image: postgres:13

environment:

- POSTGRES_USER=dvarpala

- POSTGRES_PASSWORD=password

- POSTGRES_DB=dvarpala

volumes:

- postgres_data:/var/lib/postgresql/data

- ./scripts/migration/001_initial_schema.sql:/docker-entrypoint-initdb.d/01_

ports:

- "5432:5432"

redis:

image: redis:6-alpine

ports:

- "6379:6379"

volumes:

- redis_data:/data

openvpn:

image: kylemanna/openvpn

cap_add:

- NET_ADMIN

devices:

- /dev/net/tun

ports:


```
    - "1194:1194/udp"
    - "7505:7505"
volumes:
  - ./config/openvpn:/etc/openvpn
  - openvpn_data:/etc/openvpn
environment:
  - OVPN_DATA=/etc/openvpn
```

```
volumes:
  postgres_data:
  redis_data:
  openvpn_data:
```

Dockerfile.web

dockerfile

deployment/docker/Dockerfile.web

FROM python:3.9-slim

Set working directory

WORKDIR /app

Install system dependencies

RUN apt-get update && apt-get install -y \
 postgresql-client \
 build-essential \
 && rm -rf /var/lib/apt/lists/*

Copy requirements and install Python dependencies

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

Copy application code

COPY . .

Create log directory

RUN mkdir -p /var/log/dvarpala

Set environment variables

ENV PYTHONPATH=/app

ENV FLASK_APP=src/app.py

Expose port

EXPOSE 5000

Health check

HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
 CMD curl -f http://localhost:5000/health || exit 1

Run application

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "--worker-class", "g

Production Kubernetes Deployment

namespace.yaml

yaml

deployment/kubernetes/namespace.yaml

apiVersion: v1

kind: Namespace

metadata:

name: dvarpala

labels:

name: dvarpala

deployment.yaml

yaml

```
# deployment/kubernetes/deployment.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: dvarpala-web
```

```
  namespace: dvarpala
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: dvarpala-web
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: dvarpala-web
```

```
    spec:
```

```
      containers:
```

```
        - name: web
```

```
          image: dvarpala/web:latest
```

```
          ports:
```

```
            - containerPort: 5000
```

```
          env:
```

```
            - name: DATABASE_URL
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: dvarpala-secrets
```

```
                  key: database-url
```

```
            - name: REDIS_URL
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: dvarpala-secrets
```

```
                  key: redis-url
```

```
            - name: SECRET_KEY
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: dvarpala-secrets
```

```
                  key: secret-key
```

```
            - name: GOOGLE_CLIENT_ID
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: oauth-secrets
```

```
                  key: google-client-id
```

```
            - name: GOOGLE_CLIENT_SECRET
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: oauth-secrets
```

```
    key: google-client-secret
resources:
  requests:
    memory: "256Mi"
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
livenessProbe:
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 5000
  initialDelaySeconds: 5
  periodSeconds: 5
```

service.yaml

yaml

```
# deployment/kubernetes/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: dvarpala-web-service
  namespace: dvarpala
spec:
  selector:
    app: dvarpala-web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
  type: ClusterIP
```

ingress.yaml

yaml

deployment/kubernetes/ingress.yaml

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: dvarpala-ingress

namespace: dvarpala

annotations:

kubernetes.io/ingress.class: nginx

cert-manager.io/cluster-issuer: letsencrypt-prod

nginx.ingress.kubernetes.io/ssl-redirect: "true"

spec:

tls:

- hosts:

- dvarpala.yourcompany.com

secretName: dvarpala-tls

rules:

- host: dvarpala.yourcompany.com

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: dvarpala-web-service

port:

number: 80

OpenVPN Server Setup

Installation Script

bash


```
#!/bin/bash
# scripts/setup/configure-openvpn.sh

set -e

echo "Setting up OpenVPN server for Dvarpala..."

# Install OpenVPN
apt-get update
apt-get install -y openvpn easy-rsa iptables-persistent

# Setup PKI
make-cadir /etc/openvpn/easy-rsa
cd /etc/openvpn/easy-rsa

# Configure PKI variables
cat > vars << EOF
set_var EASYRSA_REQ_COUNTRY    "US"
set_var EASYRSA_REQ_PROVINCE   "CA"
set_var EASYRSA_REQ_CITY       "San Francisco"
set_var EASYRSA_REQ_ORG        "YourCompany"
set_var EASYRSA_REQ_EMAIL      "admin@yourcompany.com"
set_var EASYRSA_REQ_OU         "IT Department"
set_var EASYRSA_ALGO           "ec"
set_var EASYRSA_DIGEST         "sha512"
EOF

# Initialize PKI
./easyrsa init-pki
./easyrsa build-ca nopass
./easyrsa gen-req server nopass
./easyrsa sign-req server server
./easyrsa gen-dh
openvpn --genkey --secret /etc/openvpn/easy-rsa/pki/ta.key

# Copy certificates to OpenVPN directory
cp /etc/openvpn/easy-rsa/pki/ca.crt /etc/openvpn/ssl/
cp /etc/openvpn/easy-rsa/pki/issued/server.crt /etc/openvpn/ssl/
cp /etc/openvpn/easy-rsa/pki/private/server.key /etc/openvpn/ssl/
cp /etc/openvpn/easy-rsa/pki/dh.pem /etc/openvpn/ssl/dh2048.pem
cp /etc/openvpn/easy-rsa/pki/ta.key /etc/openvpn/ssl/

# Set proper permissions
chmod 600 /etc/openvpn/ssl/server.key
chmod 600 /etc/openvpn/ssl/ta.key
```

```
# Create client configuration directory
mkdir -p /etc/openvpn/ccd

# Setup authentication scripts
cp config/openvpn/auth-scripts/* /etc/openvpn/auth-scripts/
chmod +x /etc/openvpn/auth-scripts/*.py
chmod +x /etc/openvpn/auth-scripts/*.sh

# Install Python dependencies for auth scripts
pip3 install redis pycpg2-binary requests

# Configure firewall rules
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
iptables -A INPUT -i tun+ -j ACCEPT
iptables -A FORWARD -i tun+ -j ACCEPT
iptables -A FORWARD -i tun+ -o eth0 -m state --state RELATED,ESTABLISHED -j ACCEPT
iptables -A FORWARD -i eth0 -o tun+ -m state --state RELATED,ESTABLISHED -j ACCEPT

# Save iptables rules
iptables-save > /etc/iptables/rules.v4

# Enable IP forwarding
echo 'net.ipv4.ip_forward=1' >> /etc/sysctl.conf
sysctl -p

# Start and enable OpenVPN
systemctl start openvpn@server
systemctl enable openvpn@server

echo "OpenVPN server setup complete!"
```

Client Configuration Generator

bash

```
#!/bin/bash
# scripts/setup/generate-client-config.sh

CLIENT_NAME="dvarpala-client"
OUTPUT_DIR="/etc/openvpn/client-configs"
SERVER_IP="your-vpn-server.com"

mkdir -p $OUTPUT_DIR

cat > $OUTPUT_DIR/$CLIENT_NAME.ovpn << EOF
client
dev tun
proto udp
remote $SERVER_IP 1194
resolv-retry infinite
nobind
auth-user-pass
auth-nocache
cipher AES-256-CBC
auth SHA256
key-direction 1
verb 3
mute 20

# Embedded certificates
<ca>
$(cat /etc/openvpn/ssl/ca.crt)
</ca>

<cert>
$(cat /etc/openvpn/ssl/client.crt)
</cert>

<key>
$(cat /etc/openvpn/ssl/client.key)
</key>

<tls-auth>
$(cat /etc/openvpn/ssl/ta.key)
</tls-auth>
EOF

# Create auth file
cat > $OUTPUT_DIR/auth.txt << EOF
temp_user
temp_portal_access
```

EOF

```
echo "Client configuration generated: $OUTPUT_DIR/$CLIENT_NAME.ovpn"
echo "Distribute this file to users along with auth.txt"
```

Quick Start Commands

Project Initialization

```
bash

# Create project directory
mkdir dvarpala && cd dvarpala
git init

# Create directory structure
mkdir -p src/{core,models,auth/oauth,vpn,access_control,api/v1,web/{auth,admin,user}}
mkdir -p templates/{auth,admin/{users,groups,resources,permissions},user,components}
mkdir -p static/{css,js/{admin,auth,common},images,fonts}
mkdir -p tests/{unit,integration,e2e,fixtures}
mkdir -p config/{environments,openvpn/auth-scripts,nginx/sites-available}
mkdir -p scripts/{setup,migration,deployment,monitoring}
mkdir -p docs/{api,deployment,architecture,user-guides}
mkdir -p deployment/{docker,kubernetes,terraform/modules/{vpc,rds,eks},ansible/{playbooks}}
mkdir -p monitoring/{prometheus/rules,grafana/{dashboards,provisioning},logs}
mkdir -p tools/{linting,security,development}

# Initialize Python packages
find src -type d -exec touch {}/__init__.py \;
find tests -type d -exec touch {}/__init__.py \;

# Create initial files
touch requirements.txt setup.py Makefile docker-compose.yml
touch .env.example .gitignore README.md
```

Development Environment Setup

```
bash
```

```
# Install Python dependencies
```

```
pip install -r requirements.txt
```

```
# Setup database
```

```
createdb dvarpala
```

```
psql dvarpala -f scripts/migration/001_initial_schema.sql
```

```
# Start Redis
```

```
redis-server
```

```
# Setup environment variables
```

```
cp .env.example .env
```

```
# Edit .env with your configuration
```

```
# Run database migrations
```

```
alembic upgrade head
```

```
# Create admin user
```

```
python -c "
```

```
from src.services.user_service import UserService
```

```
admin = UserService.create_user(
```

```
    email='admin@yourcompany.com',
```

```
    full_name='Admin User',
```

```
    department='IT',
```

```
    oauth_provider='gmail'
```

```
)
```

```
print(f'Admin user created: {admin.email}')
```

```
"
```

```
# Start development server
```

```
flask run --debug
```

Docker Development Setup

```
bash
```

```
# Build and start services
```

```
docker-compose up --build
```

```
# Setup database in container
```

```
docker-compose exec web python scripts/migration/migrate.py
```

```
# Create test data
```

```
docker-compose exec web python tools/development/seed_data.py
```

```
# View logs
```

```
docker-compose logs -f web
```

Testing Commands

```
bash
```

```
# Run unit tests
```

```
pytest tests/unit -v
```

```
# Run integration tests
```

```
pytest tests/integration -v
```

```
# Run all tests with coverage
```

```
pytest --cov=src --cov-report=html
```

```
# Run security tests
```

```
bandit -r src/
```

```
safety check
```

```
# Run linting
```

```
flake8 src/
```

```
pylint src/
```

Deployment Commands

```
bash
```

```
# Production deployment
```

```
./scripts/deployment/deploy.sh production
```

```
# Kubernetes deployment
```

```
kubectl apply -f deployment/kubernetes/
```

```
# Check deployment status
```

```
kubectl get pods -n dvarpala
```

```
kubectl logs -f deployment/dvarpala-web -n dvarpala
```

```
# Database backup
```

```
./scripts/deployment/backup.sh
```

```
# Rollback deployment
```

```
./scripts/deployment/rollback.sh
```

CLI Management Commands

```
bash
```

```
# User management
```

```
python -m src.cli user create --email john@company.com --name "John Doe" --departm
```

```
python -m src.cli user list --status active
```

```
python -m src.cli user deactivate --email john@company.com
```

```
# Group management
```

```
python -m src.cli group create --name developers --description "Software develop
```

```
python -m src.cli group assign --user john@company.com --group developers
```

```
# VPN management
```

```
python -m src.cli vpn sessions --active
```

```
python -m src.cli vpn disconnect --user john@company.com
```

```
python -m src.cli vpn stats
```

```
# Admin commands
```

```
python -m src.cli admin create-superuser --email admin@company.com
```

```
python -m src.cli admin sync-groups --from-ldap
```

```
python -m src.cli admin audit-report --days 30
```

Future Enhancements

Phase 1: Core Features (MVP)

-  Basic VPN authentication flow

-  OAuth integration (Google/Microsoft/GitHub)
-  User and group management
-  Admin dashboard
-  Basic access control

Phase 2: Advanced Access Control

- **Device Trust:** MAC address binding and device certificates
- **Conditional Access:** Location-based access policies
- **Time-based Access:** Scheduled access windows
- **Resource-level Permissions:** Fine-grained access control

Phase 3: Enterprise Features

- **LDAP/Active Directory Integration:** Sync users and groups
- **SAML SSO:** Enterprise SSO support
- **Multi-factor Authentication:** Hardware tokens, SMS, authenticator apps
- **Compliance Reporting:** SOX, HIPAA, GDPR compliance reports

Phase 4: Advanced Security

- **Zero Trust Networking:** Micro-segmentation and policy enforcement
- **Behavioral Analytics:** ML-based anomaly detection
- **Privileged Access Management:** Just-in-time access for admin operations
- **Threat Intelligence:** Integration with security feeds

Phase 5: Scalability & Performance

- **Multi-region Deployment:** Global VPN server distribution
- **Load Balancing:** High availability and performance optimization
- **Caching Layer:** Redis cluster for session management
- **API Rate Limiting:** Protection against abuse

Integration Roadmap

SSH Access Control Tool

python

Example integration with Dvarpala APIs

```
class SSHAccessController:
    def authorize_ssh_access(self, username, server_ip, ssh_key):
        response = requests.post('https://dvarpala.company.com/api/v1/auth/ssh/auth',
            {'email': f'{username}@company.com',
             'server_ip': server_ip,
             'ssh_key_fingerprint': ssh_key
            })

        if response.json()['authorized']:
            return True, response.json()['allowed_commands']
        return False, []
```

Database Access Control

python

PostgreSQL access control via Dvarpala

```
class DatabaseAccessController:
    def check_database_access(self, user_email, database_name, operation):
        response = requests.post('https://dvarpala.company.com/api/v1/auth/validate',
            {'email': user_email,
             'resource': f'database-{database_name}',
             'access_type': operation
            })

        return response.json()['authorized']
```

Kubernetes RBAC Integration

yaml

Example Kubernetes ClusterRole using Dvarpala groups

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: dvarpala-developers
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
```

Monitoring and Observability

Key Metrics to Track

- **VPN Connection Success Rate:** Monitor authentication failures
- **Session Duration:** Average and median session times
- **Resource Access Patterns:** Most accessed resources by group
- **Security Events:** Failed login attempts, suspicious activities
- **Performance Metrics:** API response times, database query performance

Alerting Rules

yaml

Prometheus alerting rules

groups:

- name: dvarpala-security

rules:

- alert: HighFailedAuthRate

expr: rate(dvarpala_auth_failures_total[5m]) > 0.1

for: 2m

labels:

severity: warning

annotations:

summary: "High authentication failure rate detected"

- alert: SuspiciousLoginPattern

expr: increase(dvarpala_auth_failures_total[1h]) > 10

for: 1m

labels:

severity: critical

annotations:

summary: "Potential brute force attack detected"

Security Considerations

Data Protection

- **Encryption at Rest:** All sensitive data encrypted in database
- **Encryption in Transit:** TLS 1.3 for all communications
- **Key Management:** Hardware Security Modules (HSM) for production
- **Audit Logging:** Immutable audit trail for all access events

Compliance Framework

- **GDPR:** Data privacy and user consent management
- **SOX:** Financial controls and access logging
- **HIPAA:** Healthcare data protection
- **ISO 27001:** Information security management

Incident Response

- **Automated Response:** Immediate session termination on security events
 - **Threat Hunting:** Proactive security monitoring
 - **Forensics:** Detailed logging for security investigations
 - **Recovery Procedures:** Disaster recovery and business continuity
-

Conclusion

The Dvarpala VPN project provides a comprehensive, enterprise-grade solution for secure network access control. By implementing Zero Trust principles with OAuth-based authentication and group-based permissions, it ensures that only authorized users can access organizational resources.

The modular architecture allows for scalable development and easy integration with existing enterprise tools. The project structure supports both rapid prototyping and production deployment, with comprehensive testing, monitoring, and security features built-in.

Key benefits of the Dvarpala approach:

- **Enhanced Security:** Multi-factor authentication with real-time access validation
- **Centralized Management:** Single point of control for all network access
- **Scalable Architecture:** Supports growth from small teams to enterprise scale
- **API-First Design:** Easy integration with existing tools and services
- **Compliance Ready:** Built-in audit logging and security controls

This reference document serves as the complete guide for implementing, deploying, and maintaining the Dvarpala VPN system, providing all necessary technical details and best practices for a successful implementation.